

SothisAI 2.9

FAQ

DAWNING INFORMATION INDUSTRY CO.,LTD.

声明

本手册的用途在于帮助您正确地使用曙光公司软件产品(以下称“本产品”), 在使用过程中出现的常见问题和解决方法。在安装和第一次使用本产品前, 请您务必先仔细阅读随机配送的所有资料, 特别是在使用本产品时遇到问题时, 本文档会有助于您更好更快地解决问题。请妥善保管本手册, 以便日后参阅。

如您不正确地或未按本手册的指示解决问题, 或让非曙光公司授权的技术人员修理、变更本产品, 曙光公司将不对由此导致的损害承担任何责任。

本手册中所提供照片、图形、图表和插图, 仅用于解释和说明目的, 可能与实际产品有些差别, 另外, 产品实际规格和配置可能会根据需求不时变更, 因此可能与本手册内容有所不同。请以实际产品为准。

本手册中所提及的非曙光公司网站信息, 是为了方便起见而提供, 此类网站中的信息不是曙光公司产品资料的一部分, 也不是曙光公司服务的一部分, 曙光公司对这些网站及信息的准确性和可用性不做任何保证。使用此类网站带来的风险将由您自行承担。

本手册不用于表明曙光公司对其产品和服务做了任何保证, 无论是明示的还是默示的(包括但不限于本手册中推荐使用产品的适用性、安全性、适销性和适合某特定用途的保证)。对本产品及相关服务的保证和保修承诺, 应按可适用的协议或产品标准保修服务条款和条件执行。在法律法规的最大允许范围内, 曙光公司对于您的使用或不能使用本产品而发生的任何损害(包括但不限于直接或间接的个人损害、商业利润的损失、业务中断、商业信息的遗失或任何其他损失), 不负任何赔偿责任。

对于您在本产品之外使用本产品提供的软件, 或经曙光公司认证推荐使用的专用软件之外的其他软件, 曙光公司对其可靠性不做任何保证。

曙光公司已经对本手册进行了仔细的校勘和核对, 但不能保证本手册完全没有任何错误和疏漏。为更好地提供服务, 曙光公司可能会对本手册中描述的产品软件和硬件及本手册的内容随时进行更正或改进, 恕不另行通知。如果您在使用过程中发现本产品的实际情况与本手册有不一致之处, 或您想得到最新的信息或有任何问题和想法, 欢迎致电 400-810-0466 或登录曙光公司服务网站 www.sugon.com 垂询。

商标和版权

“SUGON”及图标是曙光信息产业股份有限公司的商标或注册商标。

“中科曙光”及图标是曙光信息产业股份有限公司的商标或注册商标，文中“曙光信息产业股份有限公司”简称“曙光公司”。

“Intel”、“Xeon”图标是 Intel 公司的注册商标。

“Microsoft”、“Windows”、“Windows Server”及“Windows Server System”是微软公司的商标或注册商标。

上面未列明的本手册提及的其他产品、标志和商标名称也可能是其他公司的商标或注册商标，并由其各自公司、其他性质的机构或个人拥有。

在本用户手册中描述的随机软件，是基于最终用户许可协议的条款和条件提供的，只能按照最终用户许可协议的规定使用和复制。

版权所有©2023 曙光信息产业股份有限公司，所有权利保留。

本手册受到著作权法律法规保护，未经曙光信息产业股份有限公司事先书面授权，任何人士不得以任何方式对本手册的全部或任何部分进行复制、抄录、删减或将其编译为机读格式，以任何形式在可检索系统中存储，在有线或无线网络中传输，或以任何形式翻译为任何文字。

目录

声明.....	ii
商标和版权	iii
目录.....	1
1 用户指南.....	3
2 常见问题与解答	3
2.1 SothisAI 平台安装和维护	3
2.1.1 在 Ubuntu 版本过程中提示 “source: not found”，如何处理？	3
2.1.2 为什么 MySQL（服务 my_mysql）无法启动？	3
2.1.3 为什么使用 node_list 文件安装作业调度时，只有部分节点安装成功？	4
2.1.4 系统安装过 SothisAI，卸载旧版本再次安装后提交任务状态不运行	4
2.1.5 在启用防火墙的情况下，应开放哪些端口，从而保证服务的正常访问？	5
2.1.6 如何手动安装镜像库（Harbor）？	5
2.1.7 TensorFlow 推理任务，选择模型后，无法自动解析签名。	6
2.1.8 使用 NFS 实现共享存储时，共享目录下文件所有者为 nfsnobody.....	7
2.1.9 创建容器，测试 GPU 实例不可用.....	7
2.1.10 镜像库节点或 Docker 重启后 Harbor 服务异常如何处理？	8
2.1.11 安装过程中，中断了安装程序，怎样继续安装？	9
2.1.12 若先卸载了管理节点，计算节点的组件应如何卸载.....	9
2.1.13 若增量安装计算节点过程中出现“fatal: read_slurm_conf: hostnames inconsistency detected”，如何处理.....	10
2.2 普通用户使用常见问题	10
2.2.1 容器内 apt、yum 安装应用提示 “权限不足”	10
2.2.2 为用户开通代理后，仍然无法与公网交互.....	11
2.2.3 Notebook 服务打开异常.....	12
2.2.4 为用户开通代理后，Rstudio 模块安装模块与公网交互异常.....	16
2.2.5 SSH 连接容器提示 “ssh 连接失败，请检查容器网络是否正常”	17
2.2.6 通过后台导入镜像至 “镜像管理” 模块.....	20

2.2.7 无法加载镜像标识的深度学习模块	22
2.2.8 镜像构建失败.....	23
2.2.9 为什么分布式任务运行失败？查看容器日志提示“查看日志失败，无法找到日志文件！”	23
2.2.10 用户登录时，登录界面一直显示 Loading...?	23
2.2.11 为什么提交的任务状态为 waiting 或 failed?	24
2.2.12 为什么在镜像管理模块删除镜像后，docker image 还可以看到删除后的镜像?	24
2.2.13 “文件管理”无法使用，提示“确认是否已配置文件管理服务”？	25
2.2.14 为什么训练任务失败原因提示 OOMKilled?	25
2.2.15 为什么运行任务时报错：“No locks available”？	26
2.2.16 从先进计算 AI 服务，访问资源、账户等，提示“您还没登录或会话已超时”	26
2.3 配置说明	26
2.3.1 用户资源配额.....	26
2.3.2 单任务最大可用资源限制.....	27
2.3.3 如何调整 Pytorch、TensorFlow 等框架任务默认的 CPU 和 GPU 比例。	27
2.3.4 用户认证配置.....	28
2.3.5 Docker 容器对外提供服务	28
2.3.6 容器内挂载目录配置.....	32
2.3.7 调整镜像构建节点.....	33

1 用户指南

本文档用于描述安装和使用 SothisAI 2.9 及以上系列产品过程中遇到的常见问题，以及处理方法。

2 常见问题与解答

2.1 SothisAI 平台安装和维护

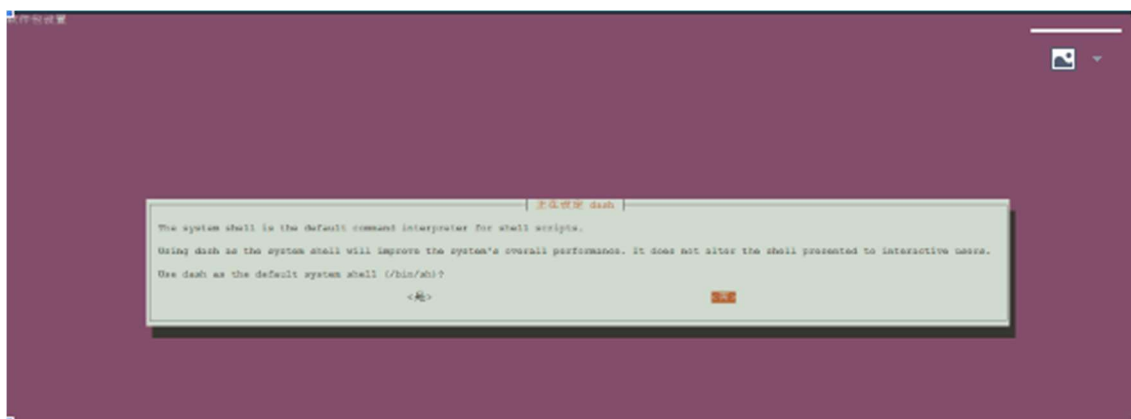
2.1.1 在 Ubuntu 版本过程中提示 “source: not found” ，如何处理？

原因是 Shell 的解释器不是 bash，需把 Shell 的解释器更改为 bash。

解决方案为是执行 `dpkg-reconfigure dash` 命令。

```
dpkg-reconfigure dash
```

然后选择 no。



2.1.2 为什么 MySQL（服务 my_mysqld）无法启动？

确认 MySQL 服务所在节点是否发生过断电？如果确定，请按如下步骤操作：

1) plugin.MYD 和 host.MYD 丢失

确认方法：打开日志文件 `/opt/gvmysql/data/{hostname}.err`，确认日志的最后部分是否出现 `plugin.MYD` 和 `host.MYD` 文件找不到的报错。

处理方法：以 `mysql` 用户身份在 `/opt/gvmysql/data/mysql/` 目录下 `touch` 创建上述两个文件，然后用 `root` 用户重启 `mysql` 确认是否恢复正常。

2) InnoDB 数据与日志不匹配

确认方法：如果日志文件 `/opt/gvmysql/data/{hostname}.err` 的最后部分出现类似下面的报错则说明是该原因导致：

```
170724 11:15:24 InnoDB: Error: page 13599 log sequence number 40904901392
```

InnoDB: is in the future! Current system log sequence number 40896476709.

InnoDB: **Your database may be corrupt or you may have copied the InnoDB**

InnoDB: **tablespace but not the InnoDB log files. See**

InnoDB: <http://dev.mysql.com/doc/refman/5.5/en/forcing-innodb-recovery.html>

InnoDB: for more information.

处理方法：请参考产品文档《mysql 安装指导.doc》中关于 Mysql 备份和恢复的章节。

2.1.3 为什么使用 node_list 文件安装作业调度时，只有部分节点安装成功？

在初始安装 SothisAI 并且选择 node_list 文件安装计算节点或者使用增量安装计算节点脚本时使用 node_list 时，发现安装完成后，只有部分计算节点安装成功，一般来说，原因包括：

- 1) 计算节点到管理节点的 ssh 无密码登录不通，导致作业调度代理向管理节点注册失败

SothisAI 安装的一个前提条件是整个集群的 ssh 无密码登录配置完毕，包括管理节点到计算节点和计算节点到管理节点。作业调度代理在计算节点上编译、安装成功后，会自动通过 ssh 登录到管理节点，修改管理节点的计算节点列表并重启调度相关服务。如果计算节点到管理节点的 ssh 无密码登录不通，则会出现计算节点上的作业调度代理正常运行，但是命令看不到该节点。

- 2) 计算节点 ssh 登录出现超时

检查下 ssh 的耗时，如果时间过长需要先处理过长的时间，否则会导致安装计算节点失败。

- 3) 某些计算节点因缺 rpm 包导致编译失败

作业调度的计算代理在安装过程中，需要用到 gcc（和 g++）编译环境和某些开发包（如 pam-devel）。如果以上条件不能满足，则编译过程一定会失败。因此，安装操作系统时，建议进行完全安装，或者至少按照“开发工作站”的模式进行安装，参考安装手册。

2.1.4 系统安装过 SothisAI，卸载旧版本再次安装后提交任务状态不运行

若当前系统安装过 SothisAI 版本，安装新版本之前，需要在管理节点上执行以下操作：

- 1) 卸载 nvidia-docker

```

docker volume ls -q -f driver=nvidia-docker | xargs -r -l {} -n1 docker ps -q -a -f volume={} | xargs -r
docker rm -f

result=`rpm -qa | grep nvidia-docker`

rpm -e $result

rm -rf /usr/bin/nvidia-docker /var/lib/nvidia-docker/

```

2) 卸载 docker

```

ps -ef | grep dockerd | grep -v grep | awk '{print $2}' | kill -9

ps -ef | grep containerd | grep -v grep | awk '{print $2}' | kill -9

dockerResult=`rpm -qa | grep docker`

rpm -e $dockerResult

rm -rf /var/lib/docker

```

2.1.5 在启用防火墙的情况下，应开放哪些端口，从而保证服务的正常访问？

开启防火墙时，为保证平台及其附属服务的正常运行，需开放以下配置文件列出的端口。

【备注】：\${SothisAI_Install_Path} 为 SothisAI 的安装路径。

1) \${SothisAI_Install_Path}/.gridview/conf/webshell/conf/netty_server.properties

开放 webshell 的 websocket 协议端口 7879:

```
netty_server_port_ws=7879
```

2) \${SothisAI_Install_Path}/.gridview/conf/sothisai/webforward.properties

开放 jupyter 服务的端口 7090:

```
jupyter_url_lan=http://127.0.0.1:7090/
```

3) \${SothisAI_Install_Path}/.gridview/conf/clustermanagement/cmconf/filemanager.properties

开放 efile 服务的端口 8089:

```
efile_url_lan=http://127.0.0.1:8089/efile
```

4) 开放平台服务端口 6080:

2.1.6 如何手动安装镜像库 (Harbor) ?

1) 执行镜像导入和安装 harbor 的脚本:

```

#/public/image_splits 为镜像切片文件的存储目录

sh /opt/gridview/install/ai/applications/install_ai_harbor.sh -image_path /public/image_splits -
image_type slice

```

【备注】：若完整镜像（镜像切片聚合后的镜像文件）已事先解压至镜像库节点的 /public/SothisAI/harbor/image_data 目录，则仅安装 harbor 即可：

```
sh /opt/gridview/install/ai/applications/install_ai_harbor.sh
```

2) 检查镜像库组件

安装完成后，登录至镜像库节点通过命令检查 Harbor 是否安装成功：

```
docker-compose -f /opt/harbor/docker-compose.yml ps
```

若组件数目为 9 个，且状态均为 healthy，则代表安装正常，如下图所示：

Name	Command	State	Ports
harbor-core	/harbor/entrypoint.sh	Up (healthy)	
harbor-db	/docker-entrypoint.sh	Up (healthy)	
harbor-jobservice	/harbor/entrypoint.sh	Up (healthy)	
harbor-log	/bin/sh -c /usr/local/bin/ ...	Up (healthy)	127.0.0.1:1514->10514/tcp
harbor-portal	nginx -g daemon off;	Up (healthy)	
nginx	nginx -g daemon off;	Up (healthy)	0.0.0.0:5000->8080/tcp
redis	redis-server /etc/redis.conf	Up (healthy)	
registry	/home/harbor/entrypoint.sh	Up (healthy)	
registryctl	/home/harbor/start.sh	Up (healthy)	

3) 检查镜像库界面

通过浏览器访问 <https://harborIP:5000/>，登录 Harbor，管理员的名称为 admin，密码为：Sugon@Harbor123（harborIP 为镜像库节点的 IP 地址）



2.1.7 TensorFlow 推理任务，选择模型后，无法自动解析签名。

1) 查看 tf_serving_service 容器是否存在（默认启动在管理节点）：

```
[root@gpunode2 ~]# docker ps|grep tf_serving_service
f8e8a401cf9b      10.0.35.26:5000/admin/tfinference/tf-serving:latest    "top"          4 days ago      Up 4 days
tf_serving_service
```

2) 集群用户主目录的共享存储重新挂载后，解析模型的 TF-Serving 容器需要重新启动，否则无法读取用户主目录的内容。

```
docker stop tf_serving_service
```

【注意】：停止容器即可，再次访问 SothisAI 时，会自动启动容器。

3) 检查管理节点平台安装目录下的配置文件（如下图）：

配置文件路径

【备注】：\${SothisAI_Install_Path} 为 SothisAI 的安装路径。

\${SothisAI_Install_Path}/.gridview/conf/sothisai/containerserviceconf/tf_serving.properties

检查 container_service_mount_dirs 字段值是否为集群中共享的用户主目录（例如：/public），如果不是，请参考文件示例修改后停止容器。

```
docker stop tf_serving_service
```

2.1.8 使用 NFS 实现共享存储时，共享目录下文件所有者为 nfsnobody

NFS Server 通过 /etc/exports 共享目录时，需配置 no_root_squash 参数。否则，共享目录下所有文件的所有者将被压制为 nfsnobody。例如：

```
/public/home *(rw,no_root_squash,async,no_subtree_check)
```

2.1.9 创建容器，测试 GPU 实例不可用

1) 现象：

```
$import torch
```

```
$torch.cuda.is_available()
```

返回 false

2) 排查过程：

(1) 检查镜像是否是 GPU 镜像

(2) 检查环境变量，将 .bashrc .bash_profile 里带家目录的都去掉后，测试依然是 false

(3) 检查是否是 nvidia 驱动的问题，GPU 节点缺少 nvidia-fabric-manager 服务，装上对应的 nvidia-fabric-manager 服务后，测试正常，返回 true

3) 原因：

Ebmgn7 系列（如 A100）的卡装完驱动后，需要额外安装 nvidia-fabric-manager 服务，不然无法正常使用 GPU 实例。

4) 安装步骤：

Centos:

```
driver_version=460.91.03
```

```
yum -y install yum-utils
```

```
yum-config-manager --add-repo
```

```

https://developer.download.nvidia.com/compute/cuda/repos/rhel7/x86_64/cuda-rhel7.repo
yum install -y nvidia-fabric-manager-`${driver_version}`-1

Ubuntu:
driver_version=460.91.03
driver_version_main=$(echo $driver_version | awk -F '.' '{print $1}')
distribution=$(. /etc/os-release;echo $ID$VERSION_ID | sed -e 's/\./g')
wget
https://developer.download.nvidia.com/compute/cuda/repos/$distribution/x86_64/cuda-
$distribution.pin
mv cuda-$distribution.pin /etc/apt/preferences.d/cuda-repository-pin-600
wget
https://developer.download.nvidia.com/compute/cuda/repos/$distribution/x86_64/7fa2af80.
pub
apt-key add 7fa2af80.pub
rm 7fa2af80.pub
echo "deb
http://developer.download.nvidia.com/compute/cuda/repos/$distribution/x86_64/" | tee
/etc/apt/sources.list.d/cuda.list
apt-get update
apt-get -y install nvidia-fabricmanager-`${driver_version_main}`=`${driver_version}`-*

```

【参考文档】：<http://www.manongjc.com/detail/29-vgpubegrshixpnp.html>

2.1.10 镜像库节点或 Docker 重启后 Harbor 服务异常如何处理？

在 SothisAI2.9 及以后版本中，增加了 Harbor 服务作为系统服务的能力，便于在镜像库节点或 Docker 重启后，Harbor 服务能够自动重启，但有时候可能没有自重启成功，导致页面的镜像服务不可用。

这时候，请在镜像仓库节点，单独执行

```
docker-compose -f /opt/harbor/docker-compose.yml up -d --force-recreate
```

执行需要一定时间，待完成后，需要以下验证

1) 在镜像库节点，输入命令查看 Harbor 组件是否正常

```
docker-compose -f /opt/harbor/docker-compose.yml ps
```

```
[root@gv222 ~]# docker-compose -f /opt/harbor/docker-compose.yml ps
```

Name	Command	State	Ports
harbor-core	/harbor/entrypoint.sh	Up (healthy)	
harbor-db	/docker-entrypoint.sh	Up (healthy)	
harbor-jobservice	/harbor/entrypoint.sh	Up (healthy)	
harbor-log	/bin/sh -c /usr/local/bin/ ...	Up (healthy)	127.0.0.1:1514->10514/tcp
harbor-portal	nginx -g daemon off;	Up (healthy)	
nginx	nginx -g daemon off;	Up (healthy)	0.0.0.0:5080->8080/tcp, 0.0.0.0:5000->8443/tcp
redis	redis-server /etc/redis.conf	Up (healthy)	
registry	/home/harbor/entrypoint.sh	Up (healthy)	
registryctl	/home/harbor/start.sh	Up (healthy)	

2) 在镜像仓库主节点, 验证登录镜像仓库是否成功

```
docker login -uadmin -pSugon@Harbor123 image.ac.com:5000
```

```
[root@gv222 ~]# docker login -uadmin -pSugon@Harbor123 image.ac.com:5000
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

2.1.11 安装过程中, 中断了安装程序, 怎样继续安装?

若在安装完成前中断了安装过程, 需要停止关键服务, 并删除如下目录及文件后再进行安装:

1) 停止平台关键服务:

```
service gridview_platform stop ; service flanneld stop ; service docker stop ; service etcd stop
```

2) 删除平台的安装目录 (假如平台安装目录为: /opt/gridview), 执行:

```
rm -rf /opt/gridview
```

3) 删除平台启动文件, 执行:

```
rm -rf /etc/init.d/gridview_platform
```

2.1.12 若先卸载了管理节点, 计算节点的组件应如何卸载

若卸载时, 先卸载了管理节点, 计算节点应参照如下流程卸载相关组件:

```
#将组件的开机启动移除
systemctl disable flannel ; system disable docker ; systemctl disable kubelet ; systemctl disable
slurmd

#停止组件的服务
systemctl stop flannel ; system stop docker ; systemctl stop kubelet ; systemctl stop slurmd

#删除 docker 组件
rm -rf /usr/bin/containerd /usr/bin/containerd-shim /usr/bin/ctr /usr/bin/docker
/usr/bin/dockerd /usr/bin/docker-init /usr/bin/docker-proxy /etc/docker/daemon.json
/usr/lib/systemd/system/docker.service /var/lib/docker*
```



```
#删除 flanneld 组件
rm -rf /usr/bin/mk-docker-opts.sh /usr/bin/flanneld /usr/lib/systemd/system/flanneld.service
/run/flannel /etc/sysconfig/flannel

#删除 k8s 组件
rm -rf /root/.kube/ /etc/kubernetes/ /usr/lib/systemd/system/kube* /var/lib/kube* /usr/bin/kube*
/etc/cni /opt/cni /etc/sysconfig/flannel

#删除 slurm 组件（若平台安装在/opt/gridview 目录下）
rm -rf /opt/gridview
```

2.1.13 若增量安装计算节点过程中出现“fatal: read_slurm_conf: hostnames inconsistency detected”，如何处理

```
directing to /bin/systemctl status slurmctld.service
slurmctld.service - Slurm controller daemon
Loaded: loaded (/usr/lib/systemd/system/slurmctld.service; enabled; vendor preset: disabled)
Active: failed (Result: exit-code) since Fri 2023-07-21 10:05:41 CST; 89ms ago
Process: 6406 ExecStart=/opt/gridview/slurm/sbin/slurmctld $SLURMCTLD_OPTIONS (code=exited, status=0/SUCCESS)
Main PID: 6408 (code=exited, status=1/FAILURE)

21 09:58:41 h03r3n04 systemd[1]: Starting Slurm controller daemon...
21 09:58:41 h03r3n04 systemd[1]: PID file /var/run/slurmctld.pid not readable (yet?) after start.
21 09:58:41 h03r3n04 systemd[1]: Started Slurm controller daemon.
21 10:05:41 h03r3n04 slurmctld[6408]: fatal: read_slurm_conf: hostnames inconsistency detected
21 10:05:41 h03r3n04 systemd[1]: slurmctld.service: main process exited, code=exited, status=1/FAILURE
21 10:05:41 h03r3n04 systemd[1]: Unit slurmctld.service entered failed state.
21 10:05:41 h03r3n04 systemd[1]: slurmctld.service failed.
[Jul 21 10:05:41 CST 2023] slurmctld error, restart ...
directing to /bin/systemctl restart slurmctld.service
```

若安装过程中，在终端出现如上图红色字体信息，此信息出现原因是由于在 slurmctld 在正常运行时，增量安装脚本将新的节点写入到 slurm.conf 中并重启 slurmctld，slurmctld 检测到 slurm.conf 与 slurmctld 运行过程中的中间状态不一致导致。这并不影响增量安装计算节点流程，无需特别进行处理。

2.2 普通用户使用常见问题

2.2.1 容器内 apt、yum 安装应用提示“权限不足”

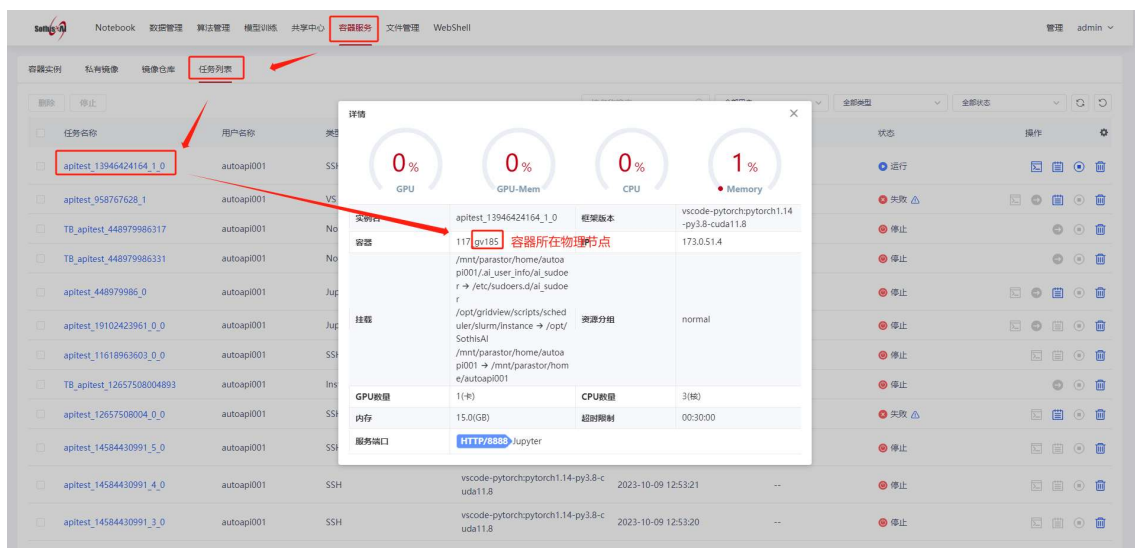
为保证平台用户在容器内生成的文件属于用户，因此登录容器的默认用户为平台用户。但为了可以使用户具备重构镜像的能力（安装应用、编译程序等）。平台允许普通用户在容器内通过 sudo 提权至 root 进行特权操作。例如：

```
#容器内切换至 root 用户
sudo su

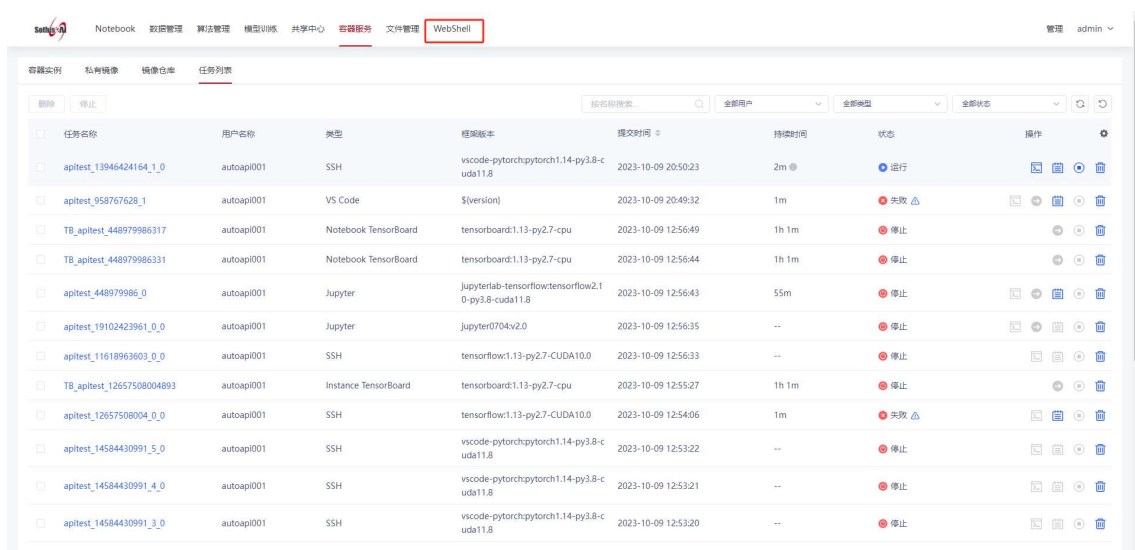
#容器内提权至 root 安装 dos2unix 软件
sudo pip install dos2unix
```

2.2.2 为用户开通代理后，仍然无法与公网交互

通过管理员登录至平台，进入任务列表模块，找到对应的任务，并点击任务名称，进入详情，查看容器所在的物理节点。



通过 webshell，登录至管理节点后台，如下图所示。



跳转至容器所在的计算节点，查看域名解析配置（/etc/resolv.conf），正常情况下，应具备“nameserver”相关的配置，如缺失相关信息，需联系集群管理员为计算节点统一配置。

```
WebShell
[root@vjob01 ~]# ssh a02in07
Last login: Mon Dec 11 15:36:54 2022 from vjob01
[root@a02in07 ~]# cat /etc/resolv.conf
nameserver 8.8.8.8
nameserver 114.114.114.114
[root@a02in07 ~]#
```

→ 跳转至计算节点

→ 查看域名解析配置

2.2.3 Notebook 服务打开异常

通过管理员登录至平台，进入任务列表模块，找到对应的任务，并点击任务名称，进入详情，查看容器所在的物理节点。

The screenshot shows the platform interface with the '任务列表' (Task List) tab selected. A task named 'apitest_13946424164_1_0' is highlighted. A modal window titled '详情' (Details) is open, showing the task's status and configuration. The modal includes a table with the following data:

资源	容器	容器版本
GPU	117	apitest_13946424164_1_0
GPU-Mem	117	apitest_13946424164_1_0
CPU	117	apitest_13946424164_1_0
Memory	117	apitest_13946424164_1_0

The modal also displays the task's configuration, including the image 'vscodc-pytorch/pytorch1.14-py3.8-cuda11.8', the command 'r → /etc/sudoers.d/ai_sudoer', and the resource group 'normal'. The task's status is '运行' (Running).

通过 webshell，登录至管理节点后台。

任务名称	用户名称	类型	镜像版本	提交时间	持续时间	状态	操作
apitest_13946424164_1_0	autoapi001	SSH	vscode-pytorchpytorch1.14-py3.8-cuda11.8	2023-10-09 20:50:23	2m @	运行	[Icons]
apitest_958767628_1	autoapi001	VS Code	\$(version)	2023-10-09 20:49:32	1m	失败	[Icons]
TB_apitest_448979986317	autoapi001	Notebook TensorBoard	tensorflow:1.13-py2.7-cpu	2023-10-09 12:56:49	1h 1m	停止	[Icons]
TB_apitest_44897998631	autoapi001	Notebook TensorBoard	tensorflow:1.13-py2.7-cpu	2023-10-09 12:56:44	1h 1m	停止	[Icons]
apitest_448979986_0	autoapi001	Jupyter	jupyterlab-tensorflowtensorflow2.10-py3.8-cuda11.8	2023-10-09 12:56:43	55m	停止	[Icons]
apitest_19102423961_0_0	autoapi001	Jupyter	jupyter0704v2.0	2023-10-09 12:56:35	--	停止	[Icons]
apitest_11618963603_0_0	autoapi001	SSH	tensorflow:1.13-py2.7-CUDA10.0	2023-10-09 12:56:33	--	停止	[Icons]
TB_apitest_12657508004893	autoapi001	Instance TensorBoard	tensorflow:1.13-py2.7-cpu	2023-10-09 12:55:27	1h 1m	停止	[Icons]
apitest_12657508004_0_0	autoapi001	SSH	tensorflow:1.13-py2.7-CUDA10.0	2023-10-09 12:54:06	1m	失败	[Icons]
apitest_14584430991_5_0	autoapi001	SSH	vscode-pytorchpytorch1.14-py3.8-cuda11.8	2023-10-09 12:53:22	--	停止	[Icons]
apitest_14584430991_4_0	autoapi001	SSH	vscode-pytorchpytorch1.14-py3.8-cuda11.8	2023-10-09 12:53:21	--	停止	[Icons]
apitest_14584430991_3_0	autoapi001	SSH	vscode-pytorchpytorch1.14-py3.8-cuda11.8	2023-10-09 12:53:20	--	停止	[Icons]

1) 校验 iptables 规则。跳转至计算节点，通过“iptables -L”指令查看 iptables 规则，若 FORWARD 规则为 DROP，需调整策略为 ACCEPT，执行：

```
iptables -P FORWARD ACCEPT
```

```

[root@02rln07 ~]# iptables -L
Chain INPUT (policy ACCEPT)
target prot opt source destination

Chain FORWARD (policy DROP)
target prot opt source destination
DOCKER-USER all -- anywhere anywhere
DOCKER-ISOLATION-STAGE-1 all -- anywhere anywhere
ACCEPT all -- anywhere anywhere ctstate RELATED,ESTABLISHED
DOCKER all -- anywhere anywhere
ACCEPT all -- anywhere anywhere
ACCEPT all -- anywhere anywhere

Chain OUTPUT (policy ACCEPT)
target prot opt source destination

Chain DOCKER (1 references)
target prot opt source destination

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
target prot opt source destination
DOCKER-ISOLATION-STAGE-2 all -- anywhere anywhere
RETURN all -- anywhere anywhere

Chain DOCKER-ISOLATION-STAGE-2 (1 references)
target prot opt source destination
DROP all -- anywhere anywhere
RETURN all -- anywhere anywhere

Chain DOCKER-USER (1 references)
target prot opt source destination
RETURN all -- anywhere anywhere
[root@02rln07 ~]# iptables -P FORWARD ACCEPT
[root@02rln07 ~]#
  
```

查看iptables规则

观察FORWARD策略是否为DROP

调整FORWARD策略为ACCEPT

2) 检查计算节点网卡转发功能。跳转至计算节点，执行：

```
sysctl -a|grep net.ipv4.ip_forward
```

若不为 1，修改/etc/sysctl.conf 文件中的 net.ipv4.ip_forward 配置项，使其为 1。

3) 校验镜像内容。若上述流程均正常，通过管理员登录至容器内部。


```
WebShell
[ec474epte@ec97f498df4 ~]$ /opt/conda/bin/python /opt/conda/bin/jupyter-lab --allow-root --ip=0.0.0.0 --port=8888 --NotebookApp.token=sothisai_860d6e51c8de42cf99480a874ee73bd0 --labApp.base_url=/jupyter-form
acd/560d6e51c8de42cf99480a874ee73bd0/
-bash: /opt/conda/bin/python: No such file or directory
[ec474epte@ec97f498df4 ~]$
```

未安装相关的Notebook服务

对于不同类型的 Notebook 服务，平台规定的安装路径如下所示：

服务类型	安装路径
jupyter	/opt/conda/bin/jupyter
codeserver	/usr/lib/code-server/bin/code-server
rstudio	/usr/lib/rstudio-server/bin/rsserver

4) 检查集群健康检查程序。若上述流程均正常，检查集群健康检查目录是否存在，正常情况下，计算节点应存在/opt/gridview/scripts/sothisai/cluster_health 目录。

```
WebShell
[root@job01 ~]$ ssh a02rln07
Last login: Mon Dec 12 15:41:13 2022
[root@a02rln07 ~]$ ls /opt/gridview/scripts/sothisai/
checkPermission.sh clear_images_local.sh clear_images.sh cluster_health
[root@a02rln07 ~]$
```

集群健康检查目录

若不存在该目录，跳回至管理节点，同步集群健康检查目录至该计算节点，执行：

#\$node 为计算节点的名称
 scp /opt/gridview/scripts/sothisai/ \$node:/opt/gridview/scripts/

```
WebShell
[root@wj001 ~]# ssh a02rn07
Last login: Mon Dec 12 19:41:13 2022
[root@a02rn07 ~]# ls /opt/gridview/scripts/sothisai/
checkPermission.sh clear_images_local.sh clear_images.sh
[root@a02rn07 ~]# 退出
Connection to a02rn07 closed.
[root@wj001 ~]# scp -r /opt/gridview/scripts/sothisai/ a02rn07:/opt/gridview/scripts/
check_cluster_health.sh
function.sh
check_all_components.sh
check_etcd.sh
check_hadoop.sh
check_container_components.sh
check_nginx.sh
check_scripts.sh
checkPermission.sh
clear_images_local.sh
clear_images.sh
[root@wj001 ~]#
```

100%	3898	6.5MB/s	00:00
100%	641	53.4KB/s	00:00
100%	511	234.3KB/s	00:00
100%	270	453.9KB/s	00:00
100%	465	696.5KB/s	00:00
100%	5224	8.3MB/s	00:00
100%	3632	5.0MB/s	00:00
100%	1747	3.2MB/s	00:00
100%	1639	1.0MB/s	00:00
100%	1637	4.1MB/s	00:00
100%	3002	6.9MB/s	00:00

在管理节点执行集群健康检查程序：

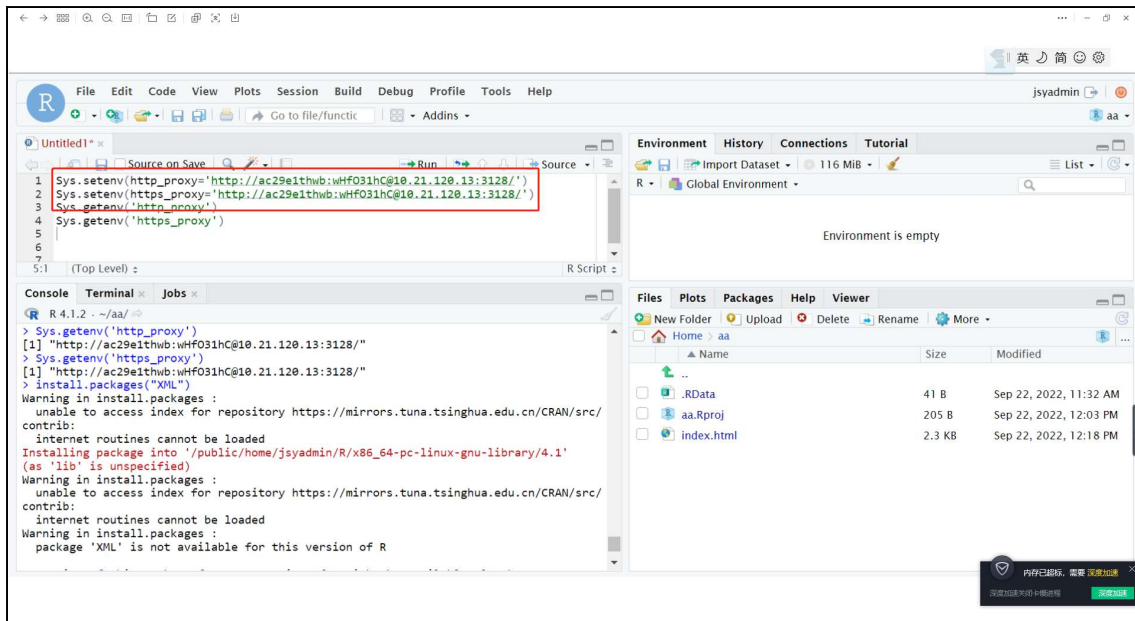
```
sh /opt/gridview/scripts/sothisai/cluster_health/check_cluster_health.sh
```

```
WebShell
[root@wj001 ~]# crontab -e
crontab: no changes made to crontab
[root@wj001 ~]# sh /opt/gridview/scripts/sothisai/cluster_health/check_cluster_health.sh
collect computer info: done use whitelist,whdnormal,whdscclu04,whdtest,whdnormal,whdscclu06,whdscclu01
[1] 19:44:12 (SUCCESS) b014n00
[2] 19:44:12 (SUCCESS) b014n01
[3] 19:44:12 (SUCCESS) b014n02
[4] 19:44:12 (SUCCESS) b014n04
[5] 19:44:12 (SUCCESS) b014n05
[6] 19:44:12 (SUCCESS) b014n06
[7] 19:44:12 (SUCCESS) b014n07
[8] 19:44:12 (SUCCESS) b014n08
[9] 19:44:12 (SUCCESS) b014n10
[10] 19:44:12 (SUCCESS) b014n11
[11] 19:44:12 (SUCCESS) b014n12
[12] 19:44:12 (SUCCESS) b014n13
[13] 19:44:12 (SUCCESS) b014n14
[14] 19:44:12 (SUCCESS) b014n15
[15] 19:44:12 (SUCCESS) b014n16
[16] 19:44:12 (SUCCESS) b014n17
[17] 19:44:12 (SUCCESS) b014n18
[18] 19:44:12 (SUCCESS) b021n00
[19] 19:44:12 (SUCCESS) b021n02
[20] 19:44:12 (SUCCESS) b021n07
[21] 19:44:12 (SUCCESS) b021n08
[22] 19:44:12 (SUCCESS) b021n09
[23] 19:44:12 (SUCCESS) b021n10
[24] 19:44:12 (SUCCESS) b021n12
[25] 19:44:12 (SUCCESS) b021n15
[26] 19:44:12 (SUCCESS) b021n17
[27] 19:44:12 (SUCCESS) b021n18
[28] 19:44:12 (SUCCESS) b021n19
[29] 19:44:12 (SUCCESS) b034n14
[30] 19:44:12 (SUCCESS) b034n15
[31] 19:44:12 (SUCCESS) b034n17
[32] 19:44:12 (SUCCESS) b034n19
[33] 19:44:12 (SUCCESS) g013n00
[34] 19:44:12 (SUCCESS) g013n01
[35] 19:44:12 (SUCCESS) g013n02
[36] 19:44:12 (SUCCESS) g013n03
[37] 19:44:12 (SUCCESS) g013n04
[38] 19:44:12 (SUCCESS) g013n05
```

2.2.4 为用户开通代理后，Rstudio 模块安装模块与公网交互异常

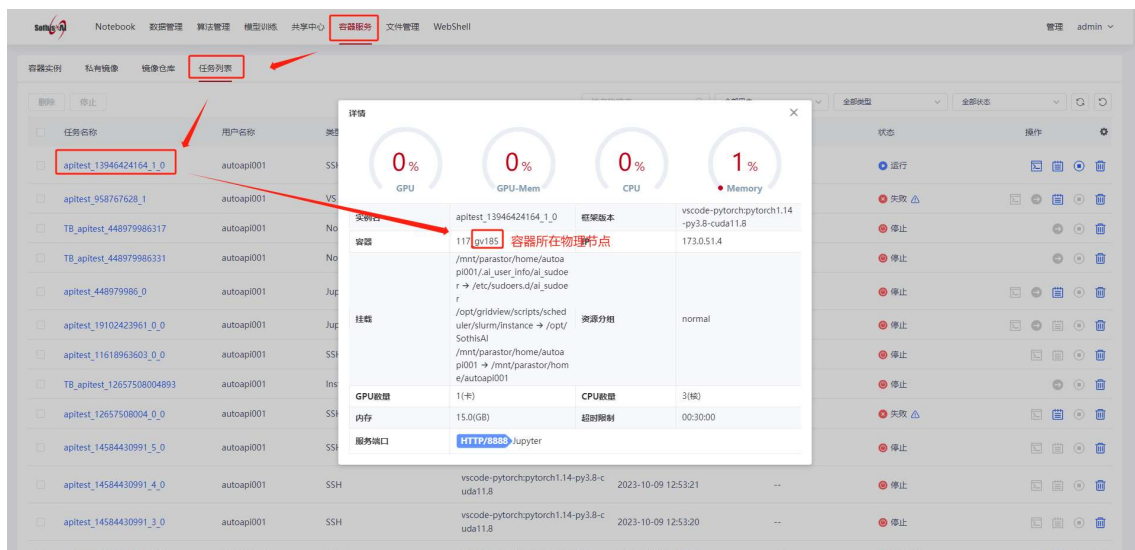
在 Rstudio 界面键入如下指令：

```
#$user 是为用户名
#$passwd 是为用户代理密码
#$proxy_ip 是为代理节点 IP
#$proxy_port 是为代理节点端口
Sys.setenv(http_proxy='http://$user:$passwd@$proxy_ip:$proxy_port/')
Sys.setenv(https_proxy='https://$user:$passwd@$proxy_ip:$proxy_port/')
```

2.2.5 SSH 连接容器提示 “ssh 连接失败，请检查容器网络是否正常”

通过管理员登录至平台，进入任务列表模块，找到对应的任务，并点击任务名称，进入详情，查看容器所在的物理节点。



通过 webshell，登录至管理节点后台。

任务名称	用户名称	类型	终端版本	提交时间	持续时间	状态	操作
apitest_13946424164_1_0	autoapi001	SSH	vscode-pytorchpytorch1.14-py3.8-cuda11.8	2023-10-09 20:50:23	2m @	运行	[Icons]
apitest_958767628_1	autoapi001	VS Code	\$(version)	2023-10-09 20:49:32	1m	失败	[Icons]
TB_apitest_448979986317	autoapi001	Notebook TensorBoard	tensorboard:1.13-py2.7-cpu	2023-10-09 12:56:49	1h 1m	停止	[Icons]
TB_apitest_44897998631	autoapi001	Notebook TensorBoard	tensorboard:1.13-py2.7-cpu	2023-10-09 12:56:44	1h 1m	停止	[Icons]
apitest_448979986_0	autoapi001	Jupyter	jupyterlab-tensorflowtensorflow2.10-py3.8-cuda11.8	2023-10-09 12:56:43	55m	停止	[Icons]
apitest_19102423961_0_0	autoapi001	Jupyter	jupyter0704v2.0	2023-10-09 12:56:35	--	停止	[Icons]
apitest_11618963603_0_0	autoapi001	SSH	tensorflow:1.13-py2.7-CUDA10.0	2023-10-09 12:56:33	--	停止	[Icons]
TB_apitest_12657508004893	autoapi001	Instance TensorBoard	tensorboard:1.13-py2.7-cpu	2023-10-09 12:55:27	1h 1m	停止	[Icons]
apitest_12657508004_0_0	autoapi001	SSH	tensorflow:1.13-py2.7-CUDA10.0	2023-10-09 12:54:06	1m	失败	[Icons]
apitest_14584430991_5_0	autoapi001	SSH	vscode-pytorchpytorch1.14-py3.8-cuda11.8	2023-10-09 12:53:22	--	停止	[Icons]
apitest_14584430991_4_0	autoapi001	SSH	vscode-pytorchpytorch1.14-py3.8-cuda11.8	2023-10-09 12:53:21	--	停止	[Icons]
apitest_14584430991_3_0	autoapi001	SSH	vscode-pytorchpytorch1.14-py3.8-cuda11.8	2023-10-09 12:53:20	--	停止	[Icons]

1) 校验 iptables 规则。跳转至计算节点，通过“iptables -L”指令查看 iptables 规则，若 FORWARD 规则为 DROP，需调整策略为 ACCEPT，执行：

```
iptables -P FORWARD ACCEPT
```

```

WebShell
[root@02rn07 ~]# iptables -L
Chain INPUT (policy ACCEPT)
target prot opt source destination

Chain FORWARD (policy DROP)
target prot opt source destination
DOCKER-USER all -- anywhere anywhere
DOCKER-ISOLATION-STAGE-1 all -- anywhere anywhere
ACCEPT all -- anywhere anywhere ctstate RELATED,ESTABLISHED
DOCKER all -- anywhere anywhere
ACCEPT all -- anywhere anywhere
ACCEPT all -- anywhere anywhere

Chain OUTPUT (policy ACCEPT)
target prot opt source destination

Chain DOCKER (1 references)
target prot opt source destination

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
target prot opt source destination
DOCKER-ISOLATION-STAGE-2 all -- anywhere anywhere
RETURN all -- anywhere anywhere

Chain DOCKER-ISOLATION-STAGE-2 (1 references)
target prot opt source destination
DROP all -- anywhere anywhere
RETURN all -- anywhere anywhere

Chain DOCKER-USER (1 references)
target prot opt source destination
RETURN all -- anywhere anywhere
[root@02rn07 ~]# iptables -P FORWARD ACCEPT
[root@02rn07 ~]#
  
```

2) 检查计算节点是否开启网卡转发功能。跳转至计算节点，执行：

```
sysctl -a|grep net.ipv4.ip_forward
```

若不为 1，修改/etc/sysctl.conf 文件中的 net.ipv4.ip_forward 配置项，使其为 1。

3) 检查集群健康检查程序。若上述流程均正常，检查集群健康检查目录是否存在，正常情况下，计算节点应存在/opt/gridview/scripts/sothisai/cluster_health 目录。

```
WebShell
[root@wj001 ~]# ssh a02rn07
Last login: Mon Dec 12 19:41:13 2022
[root@a02rn07 ~]# ls /opt/gridview/scripts/sothisai/
checkPermission.sh clear_images_local.sh clear_images.sh cluster_health
[root@a02rn07 ~]#
```

若不存在该目录，跳回至管理节点，同步集群健康检查目录至该计算节点，执行：

```
scp /opt/gridview/scripts/sothisai/ $node:/opt/gridview/scripts/
```

```
WebShell
[root@wj001 ~]# ssh a02rn07
Last login: Mon Dec 12 19:41:13 2022
[root@a02rn07 ~]# ls /opt/gridview/scripts/sothisai/
checkPermission.sh clear_images_local.sh clear_images.sh cluster_health
[root@a02rn07 ~]# 退出
Connection to a02rn07 closed.
[root@wj001 ~]# scp -r /opt/gridview/scripts/sothisai/ a02rn07:/opt/gridview/scripts/
[root@a02rn07 ~]#
function.sh
check_cluster_health.sh
check_all_components.sh
check_etcd.sh
check_harbor.sh
check_container_components.sh
check_nginx.sh
check_is_scripts.sh
checkPermission.sh
clear_images_local.sh
clear_images.sh
[root@wj001 ~]#
```

100%	3898	6.5MB/s	00:00
100%	641	53.4KB/s	00:00
100%	511	234.3KB/s	00:00
100%	270	455.3KB/s	00:00
100%	465	686.5KB/s	00:00
100%	5224	8.3MB/s	00:00
100%	3632	5.0MB/s	00:00
100%	1747	3.2MB/s	00:00
100%	1639	1.0MB/s	00:00
100%	1637	4.1MB/s	00:00
100%	3002	6.9MB/s	00:00

于管理节点执行集群健康检查程序，执行：

```
sh /opt/gridview/scripts/sothisai/cluster_health/check_cluster_health.sh
```



```
cat /opt/gridview/.gridview/conf/sothisai/scheduler.properties | egrep accelerator | grep -v '#'

accelerator=gpu
```

在 SothisAI 的规范里，镜像路径分为四级标志，第一级为加速卡类型（此处为 `gpu`），第二级为镜像所属者，第三级该镜像类型，第四级为镜像名称和标签。

其中，镜像类型的类别及含义见下表：

模块类别	功能解释
tensorflow	TensorFlow 训练
tfdataset	TensorFlow 数据预处理
tfinference	TensorFlow 推理
tftuning	TensorFlow 超参调优
pytorch	Pytorch 训练
pytorchdataset	Pytorch 数据预处理
pytorchinference	pytroch 推理
tensorboard	可视化 Tensorboard 服务
base	支持 SSH 任务
jupyterlab	支持 Jupyter 服务
codeserver	支持 CodeServer 服务
rstudio	支持 Rstudio 服务

2）导入镜像。登录至镜像库节点，通过如下指令可导入本地镜像。

```
docker load < ${image_file}
```

其中，`${image_file}` 为镜像压缩包的存储路径。例如：

```
[root@gpunode2 ~]# docker load < registry.tar
Loaded image: registry:2
[root@gpunode2 ~]#
```

3）规范镜像 URL

通过指令：`docker tag ${image_name}:${image_tag}`

`${harborVIP}:${harborPort}/${accelerator}/${user}/${type}/${image_name}:${image_tag}`

其中，

`${image_name}:${image_tag}` 为镜像导入后回显的镜像名称与标签。

`${accelerator}` 为加速器类型，目前平台仅支持 `gpu`、`dcu`、`mlu`。

`${user}` 为创建用户，若为管理员，则为 `admin`（`admin` 创建的镜像可以被平台所有用户使用）

`${type}` 为镜像类型。

例如：

```
Loaded image: registry:2
[root@gpunode2 ~]# docker tag registry:2 10.0.35.26:5000/gpu/admin/base/registry:2
[root@gpunode2 ~]#
```

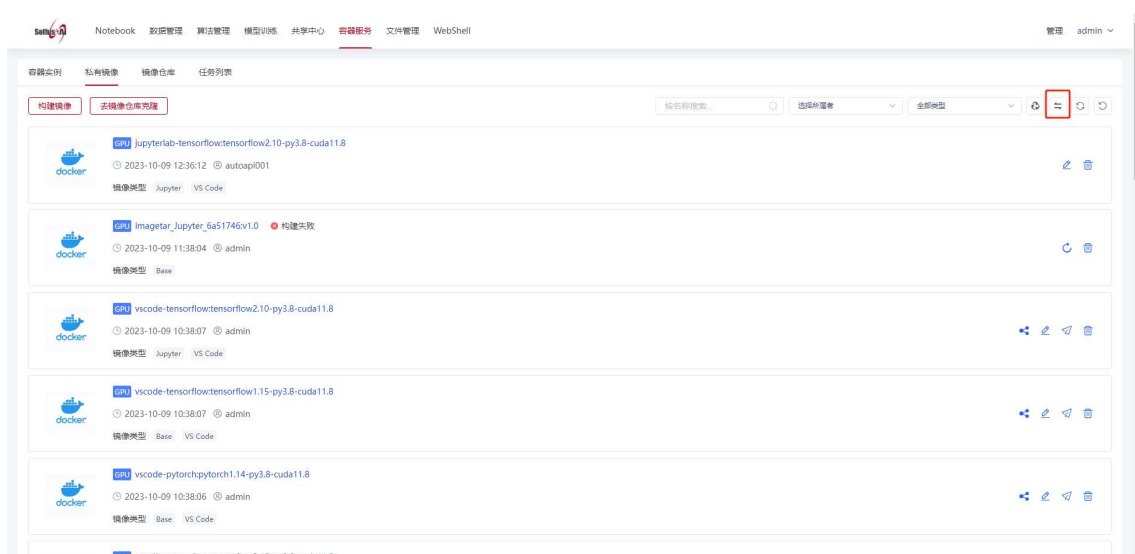
4) 推送镜像。通过下述命令将规范化 URL 后的镜像推送至镜像库。（推送前，需要先登录 Harbor，账号、密码为/opt/gridview/.gridview/conf/sothisai/image.properties 配置文件中记录的 harborUser 和 harborPasswd）

```
[root@gpunode2 ~]# docker login -uadmin -p[REDACTED] 10.0.35.26:5000
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@gpunode2 ~]# docker push 10.0.35.26:5000/gpu/admin/base/registry:2
The push refers to repository [10.0.35.26:5000/gpu/admin/base/registry]
6b263b6e9ced: Pushed
dead8a13b621: Pushed
00a8ff67f927: Pushing [=====>] 9.864MB/22.79MB
2b7bd2eefde2: Pushing [=====>] 3.325MB/5.711MB
a120b7c9a693: Pushing [=====>] 4.793MB/4.815MB
```

登录
推送

5) 同步镜像。推送完成后，在页面的“容器服务/镜像管理”模块下，点击“同步”按钮进行镜像仓库信息同步。之后可在公有库页面确认新上传的镜像信息。



2.2.7 无法加载镜像标识的深度学习模块

用户调整个人主目录下的.bashrc 文件，改变类似 PATH、Python 库函数等环境变量，导致个人环境变量、库的优先于镜像自带的模块加载。

```
WebShell
# .bashrc
# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

export PATH=/public/home/ /bin:$PATH

export ANSYSLIC_DIR=/public/home/
export WRF_DIR=/public/home/
export PATH=/public/home/ 0/bin:$PATH #2022-8-9 vasp
```

用户个人模块优先于镜像模块

需提醒用户优先加载镜像自带的模块，将个人环境变量的优先级置后。

2.2.8 镜像构建失败

平台预置了基于 CentOS6、7、8 以及 Ubuntu14、16、18、20 系统版本的 yum、apt 源素材，其他系统版本的镜像会默认使用镜像自身的源进行构建，若构建失败（镜像自带的源可能配置了集群无法解析的域名，导致构建失败），需建议用户调整基础镜像的版本或管理员协助其通过后台构建并推送至平台。

2.2.9 为什么分布式任务运行失败？查看容器日志提示“查看日志失败，无法找到日志文件！”

1) 检查 slurm 配置文件保证 PrologFlags 的值为：CONTAIN

slurm 配置文件路径

- \$SOTHAISAI_HOME/slurm/etc/slurm.conf

配置文件格式：

```
#PrologFlags=Alloc
#PrologFlags=Serial
#PrologFlags=NOHOLD
PrologFlags=CONTAIN
```

确保该配置文件中 PrologFlags=CONTAIN 内容前没有注释符“#”

2) 重启 slurmctld 服务

在 slurm 管理节点执行如下指令：

```
service slurmctld restart
```

2.2.10 用户登录时，登录界面一直显示 Loading...?

请检查管理节点挂载的共享目录能否正常访问。

2.2.11 为什么提交的任务状态为 waiting 或 failed?

1) SLURM 调度器

任务 waiting 或 failed 的提示信息：通过 `scontrol show job <job_id>` 查看 job 的失败原因。

```
[root@gv142 TF_Task_1903211585]# cat detail.job.5
JobId=5 JobName=TF_Task_1903211585
  UserId=yuki(1102) GroupId=yuki(1102) MCS_label=N/A
  Priority=4294901756 Nice=0 Account=yuki QOS=yuki_qos
  JobState=RUNNING Reason=None Dependency=(null)
  Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
  DerivedExitCode=0:0
  RunTime=00:00:00 TimeLimit=10:00:00 TimeMin=N/A
  SubmitTime=2019-03-21T17:26:09 EligibleTime=2019-03-21T17:26:09
  AccrueTime=2019-03-21T17:26:09
  StartTime=2019-03-21T17:26:09 EndTime=2019-03-22T03:26:09 Deadline=N/A
  PreemptTime=None SuspendTime=None SecsPreSuspend=0
  LastSchedEval=2019-03-21T17:26:09
  Partition=TeslaK40m AllocNode:Sid=gv142:2431
  ReqNodeList=(null) ExcNodeList=(null)
  NodeList=gpunode8
  BatchHost=gpunode8
  NumNodes=1 NumCPUs=4 NumTasks=3 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
  TRES=cpu=4,mem=6G,node=1,billing=4,gres/gpu=1
  Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
  Nodes=gpunode8 CPU_IDS=2-5 Mem=0 GRES_IDX=gpu(IDX:1)
  MinCPUsNode=3 MinMemoryNode=6G MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
  Command=/public/user/yuki/SothisAI/tensorflow/tf_task/TF_Task_1903211585/job_yuki_20190321_172609
  WorkDir=/public/user/yuki
  StdErr=/public/user/yuki/SothisAI/tensorflow/tf_task/TF_Task_1903211585/5.out
  StdIn=/dev/null
  StdOut=/public/user/yuki/SothisAI/tensorflow/tf_task/TF_Task_1903211585/5.out
  Power=
  TresPerNode=gpu:1
```

说明：任务对应的 job_id 可通过命令 `sacct -a` 查看。

2) Kubernetes 调度器

通过以下命令可以获取到：

```
kubectyl get pod tf-task-1903219280-sugon-46441-w0-n52r7 --namespace=sugon -o yaml
```

```
[root@gv209 ~]# kubectyl get pod tf-task-1903219280-sugon-46441-w0-n52r7 --namespace=sugon -o yaml | grep -A 5 -B 5 message
secretName: default-token-nvgc8
status:
  conditions:
  - lastProbeTime: null
    lastTransitionTime: 2019-03-21T07:56:29Z
    message: '0/1 nodes are available: 1 node(s) were not ready, 1 node(s) were out
      of disk space.'
    reason: Unschedulable
    status: "False"
    type: PodScheduled
  phase: Pending
[root@gv209 ~]#
```

说明：

tf-task-1903219280-sugon-46441-w0-n52r7 为 pod name，可通过下面命令获取。

```
kubectyl get pods --all-namespaces | grep <task_id>
```

2.2.12 为什么在镜像管理模块删除镜像后，docker image 还可以看到删除后的镜像？

镜像管理删除的是 Harbor（镜像仓库）中的镜像。docker image 命令查询的是该节点本地的镜像。如果需要删除该节点镜像，需要执行命令：

```
docker rmi <image id>
```

2.2.13 “文件管理” 无法使用，提示 “确认是否已配置文件管理服务” ？

- 1) 确认文件管理 IP 是否正确，如果需要修改，需要重新启动服务

- 确认 IP 是否正确

```
cat
```

```
$SOTHISAI_HOME/.gridview/conf/clustermanagement/cmconf/filemanager.properties
```

```
| grep EfileUrl
```

- 重新启动 SothisAI 的服务

```
/etc/init.d/gridview_platform restart
```

- 2) 浏览器访问，地址栏输入：

<http://IP:8089/efile/> （请替换为文件管理 IP）

- 如果不能打开 EFile 登录页（出现用户名，密码输入框），请检查端口是否被监听或被占用：

```
netstat -anlp | grep 8089
```

```
tcp6    0      0 :::8089                :::*                    LISTEN   11567/java
```

- 解决方案：
卸载并重新安装 efile。（参考《安装手册》）

2.2.14 为什么训练任务失败原因提示 OOMKilled?



一般在做模型训练的时候，数据是先读入内存，然后再按 `batchsize` 的大小分批次读入显存，做一次训练。如果内存太小，数据在读入内存的时候就可能内存不足。

- 如果提交训练任务页面可以调整内存资源申请，请增大申请的内存，再次提交任务。

- 如果页面不能自定义任务资源，需要联系管理员调整配置文件中设置的默认内存，配置文件默认路径为：

```
$SOTHISAI_HOME/.gridview/conf/sothisai/commonconf/resources.properties
```

2.2.15 为什么运行任务时报错：“No locks available” ？

```
Create train dataset..
libdc1394 error: Failed to initialize libdc1394
I0723 08:48:15.361146 11 convert_imageset.cpp:86] Shuffling data
I0723 08:49:23.690378 11 convert_imageset.cpp:89] A total of 400 images.
F0723 08:49:35.720715 11 db_imdb.hpp:15] Check failed: mdb_status == 0 (37 vs. 0) No locks available
*** Check failure stack trace: ***
@ 0x7fb1497bedaa (unknown)
@ 0x7fb1497bece4 (unknown)
@ 0x7fb1497be6e6 (unknown)
@ 0x7fb1497c1687 (unknown)
@ 0x7fb149ccba51 caffe::db::LMDB::Open()
@ 0x403a0c main
@ 0x7fb1487c6f45 (unknown)
@ 0x404c64 (unknown)
@ (nil) (unknown)
Aborted (core dumped)
Errors in create train dataset!
```

如果使用 NFS 作为共享存储，请检查 NFS 服务端和客户端节点是否开启了 `nfslock.service` 服务。

```
systemctl status nfslock.service
```

2.2.16 从先进计算 AI 服务，访问资源、账户等，提示“您还没登录或会话已超时”

- 1) 查看是否使用 chrome 浏览器 80 及以上版本；
- 2) 打开 Chrome 设置，将 `chrome://flags/#same-site-by-default-cookies` 第一个选项设置为 Disabled，然后重启浏览器。

● SameSite by default cookies

Treat cookies that don't specify a SameSite attribute as if they were SameSite=Lax. Sites must specify SameSite=None in order to enable third-party usage. – Mac, Windows, Linux, Chrome OS, Android
#same-site-by-default-cookies

Disabled

2.3 配置说明

2.3.1 用户资源配额

可在配置文件中设置用户同一时刻可用的 GPU、CPU 数量。支持默认的资源配额，也可给用户单独设置。若未针对用户设置，则默认配额生效。

配置文件路径（SothisAI 默认安装目录为 `/opt/gridview`）：

- `/opt/gridview/.gridview/conf/sothisai/scheduler.properties`

配置文件格式：

```
#User Using GPU default Limits(pcs)

gpuLimits[DEFAULT]=10

#GPU Limits, no limits if not set

gpuLimits[sugon]=5
```

【注意】：修改后，将在用户注销并重新登录平台时即可生效。

2.3.2 单任务最大可用资源限制

详见配置文件（SothisAI 默认安装目录为/opt/gridview）：

- /opt/gridview/.gridview/conf/sothisai/commonconf/resources.properties

配置文件可设置 GPU、CPU、Memory 数量和大小，不可超过物理节点硬件资源限制：

```
#The maximum number of GPUs available to a single node.
max_available_gpu_number=2
#The maximum number of CPUs available to a single node.
max_available_cpu_number=28
#The maximum size of memory available to a single node.unit:(MB)
max_available_memory_size=65536
#The maximum number of workers allowed to be created for distributed tasks.
max_available_worker_number=999
#The maximum number of PSs allowed to be created for distributed tasks.
max_available_ps_number=999
#When the disk free space of the shared directory is less than this value, no sharing is allowed.unit:(MB)
share_resource_access_threshold=1024
```

2.3.3 如何调整 Pytorch、TensorFlow 等框架任务默认的 CPU 和 GPU 比例。

详见配置文件（SothisAI 默认安装目录为/opt/gridview）：

- /opt/gridview/.gridview/conf/sothisai/tfconf/tf_config.properties

配置文件示例如下：

```

Created by JInto - www.guh-software.de
#Mon Nov 12 16:53:26 CST 2018

#TF dataset task cpu number
tf_dataset_cpu_number=1

#TF dataset task ram size,unit:(MB)
tf_dataset_ram_size=2048

#Distributed task worker cpu number
tf_distributed_worker_cpu_number=1

#Distributed task worker gpu number
tf_distributed_worker_gpu_number=1

#Distributed task worker ram size,unit:(MB)
tf_distributed_worker_ram_size=2048

#Non distributed task worker cpu number
tf_non_distributed_worker_cpu_number=2

#Non distributed task worker ram size,unit:(MB)
tf_non_distributed_worker_ram_size=4096

```

2.3.4 用户认证配置

1) 配置文件在哪？

- `${SOTHISAI_HOME}/.gridview/conf/secidentify/authconf/authMethodEnv.properties`

2) 文件中的各项配置含义

`authEnv=local` #认证类型，SothisAI 支持的类型有 `local`、`nis`，填写多个时以逗号分隔，请注意必须都是小写。

#如果 `authEnv` 中配置了 `nis`，那么就需要设置以下两个 `nis` 相关属性

`nis.server=127.0.0.1` # `nis` server IP

`nis.domainname=tnis140` # `nis` server 域名

#如果 `authEnv` 中配置了 `local`，那么就需要设置以下两个 `local` 相关属性

`local.server=127.0.0.1` #此项不需修改

`local.nodelist=SCHEDULER` #`local` 认证的节点范围，`SCHEDULER` 表示所有计算节点

2.3.5 Docker 容器对外提供服务

2.3.5.1 目的

解决容器内部对外提供服务的问题。计算节点的容器内部开放指定的端口，然后可以通过管理节点或者登录节点对外开放的端口访问到计算节点容器内部开放的指定端口。

目前仅支持 `HTTP` 和 `SOCKET` 两种协议。

2.3.5.2 服务端点配置

配置文件：

```
/opt/gridview/.gridview/conf/sothisai/webforward.properties
```

配置文件示例如下：

```
#After modifying the configuration,the platform will take effect immediately in a minute.
jupyter_url_wan=http://10.8.150.23:7090/
jupyter_url_lan=http://10.8.150.23:7090/
batch_execution_status=true
#端口类型,多个时使用,分割。label-value,label是前端展示用,value是后端实际协议
protocol_type=HTTP-HTTP
```

“protocol_type”字段的值影响支持的服务端口。目前仅支持 HTTP 和 SOCKET 两种。

可配置的值为：HTTP-HTTP、SOCKET-SSH，多个用英文逗号分割。用户退出重新登录后生效。

2.3.5.3 场景分析

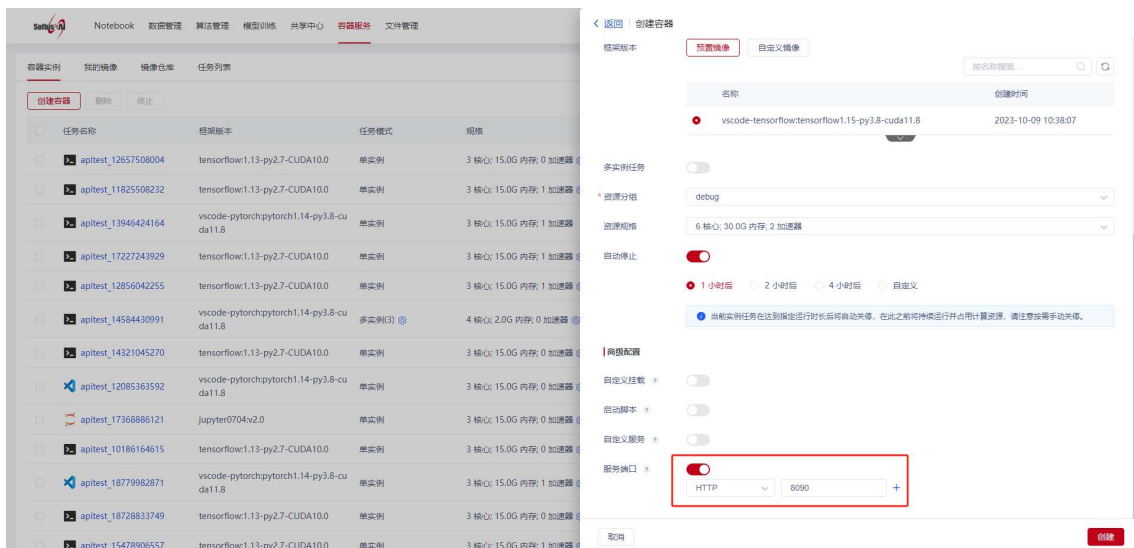
场景 1：用户在容器内部署 WEB 服务，并期望暴露至互联网，可通过特定 URL 访问：

场景 2：用户在容器内部发布 Socket 服务。

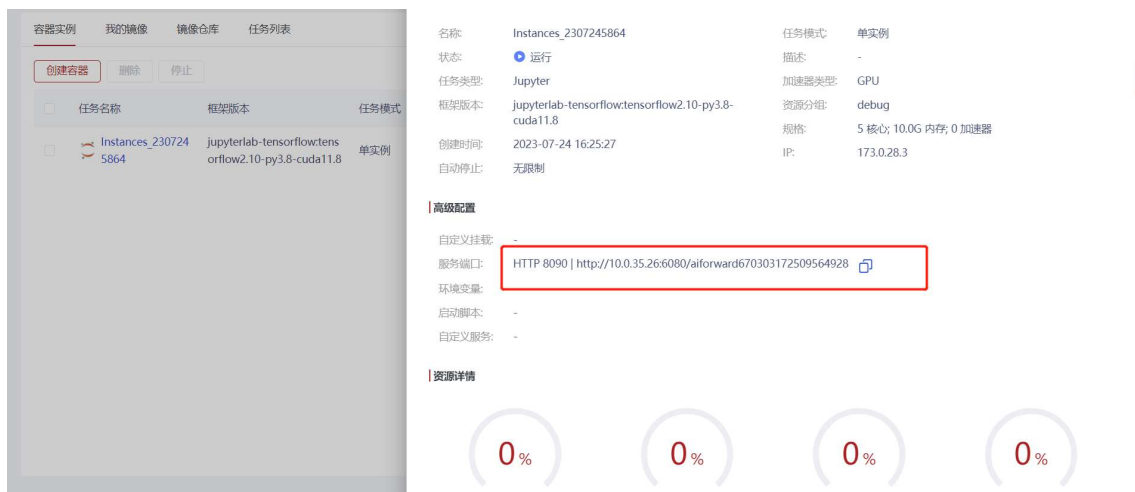
2.3.5.4 HTTP 配置

1) 管理员开启了 HTTP 服务端口。即配置文件中 protocol_type 的值存在 “HTTP-HTTP”

2) 用户在 Jupyter 面板中添加服务端口，并指定一个端口号，如 8090



3) 点击容器详情后，出现访问 URL,URL 内容为：http://ip:port/sothisai/api/containers/container-service?id=aiforward598332344578867200&resource=



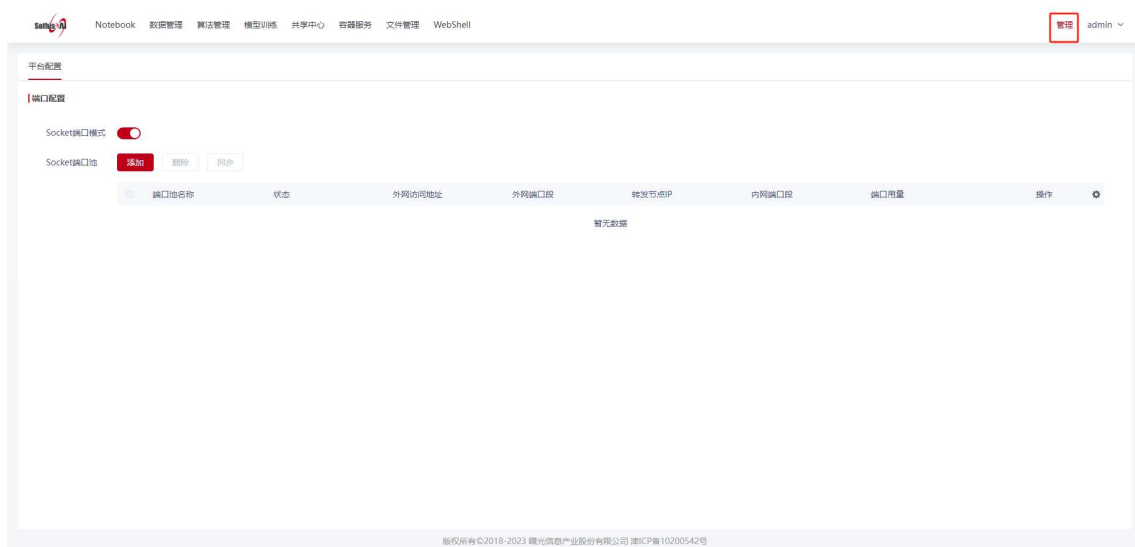
4) 将上述 URL 中 “id” 的值，添加到容器内部服务的上下文中。如 Tomcat 服务配置如下：

```
<Context path="/aiforward585793982068424704&resource" docBase="" reloadable="true"/>
```

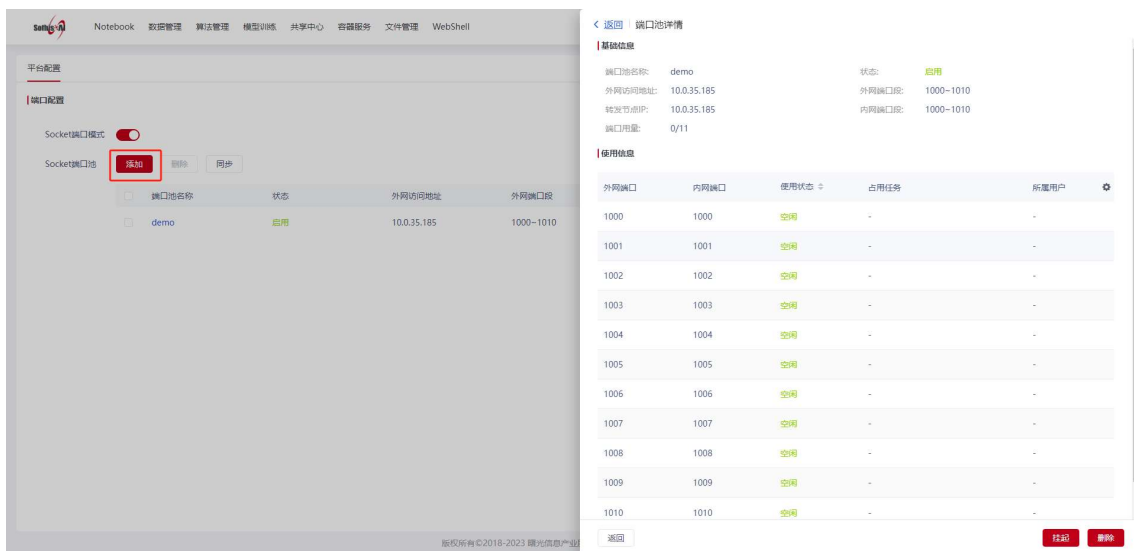
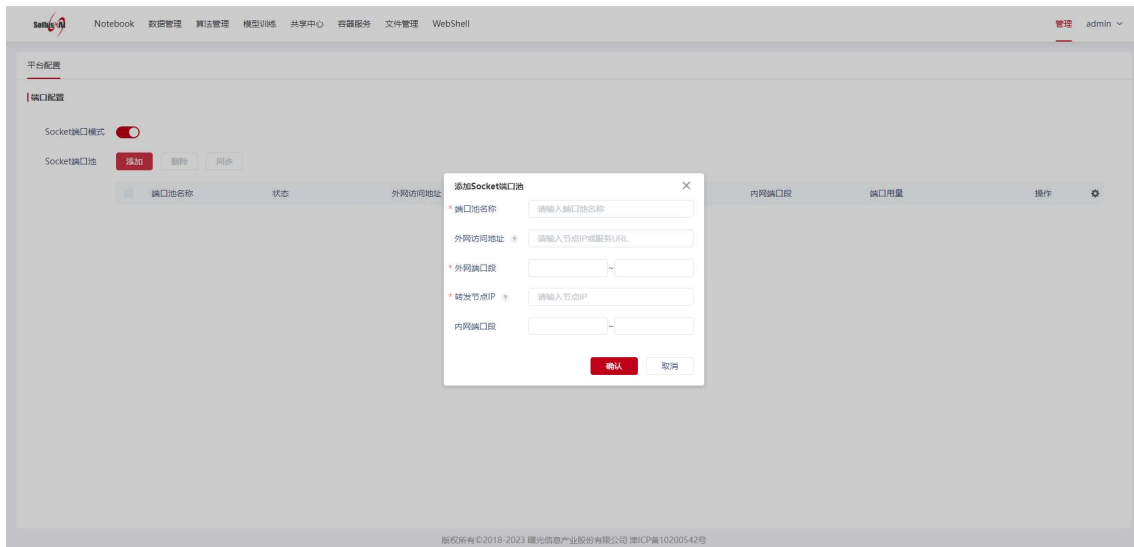
5) 访问上述 URL，即可访问到容器内部服务。

2.3.5.5 SOCKET 配置

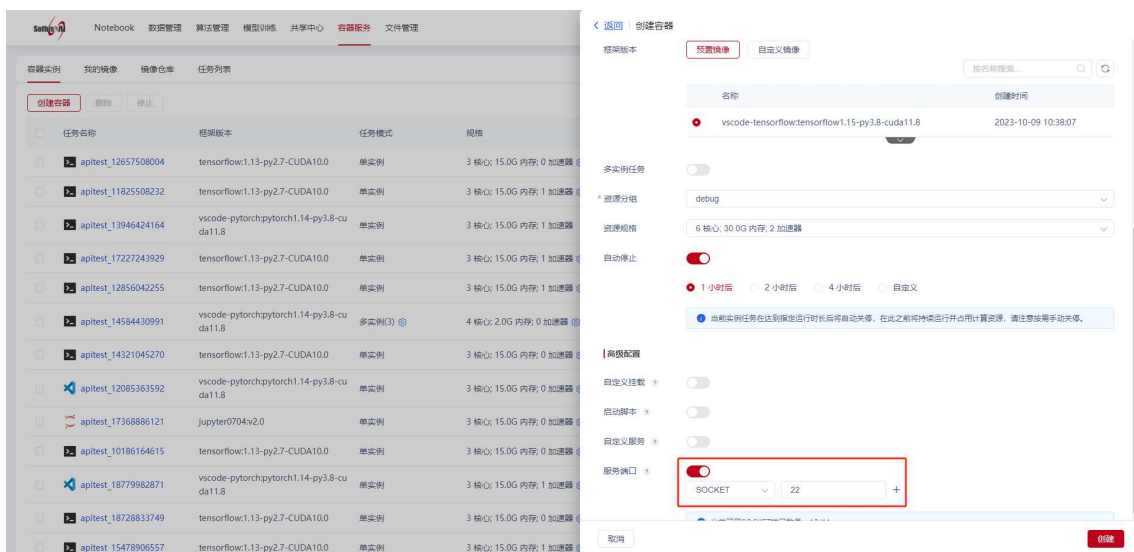
1、AI 管理员 admin 登录到平台，点击管理-平台配置开启 socket 端口模式开关



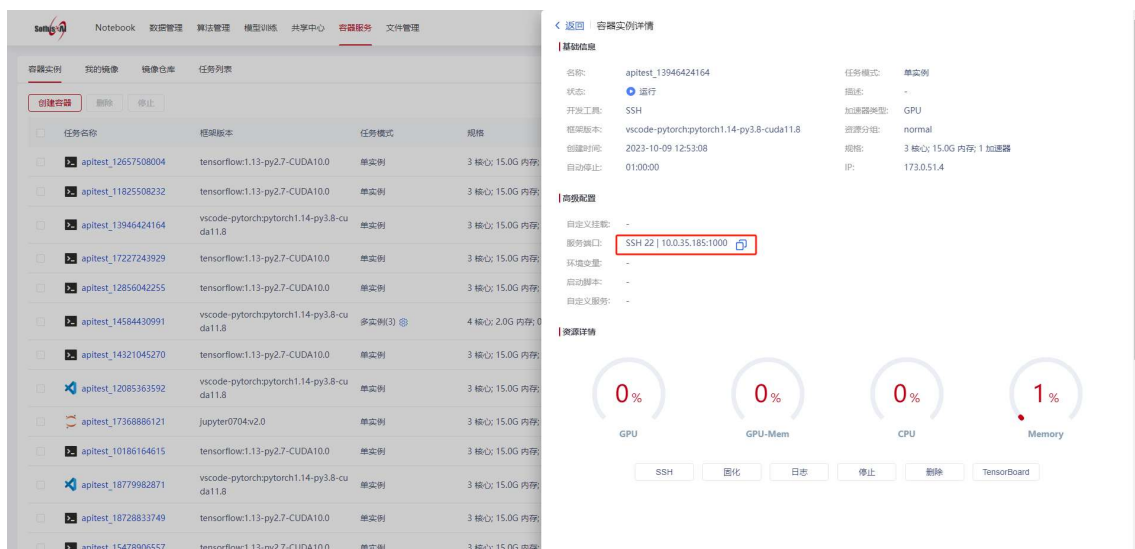
2、添加端口池信息



3、用户在容器服务模块创建容器时指定 socket 端口模式，设定容器内需要对外映射的端口信息



4、任务创建成功后，点击任务详情即可看到映射后的地址



5、通过映射后的地址 10.0.35.185:1000 和当前登录用户即可通过连接到容器内部的 22 端口

2.3.6 容器内挂载目录配置

1) 配置文件在哪？

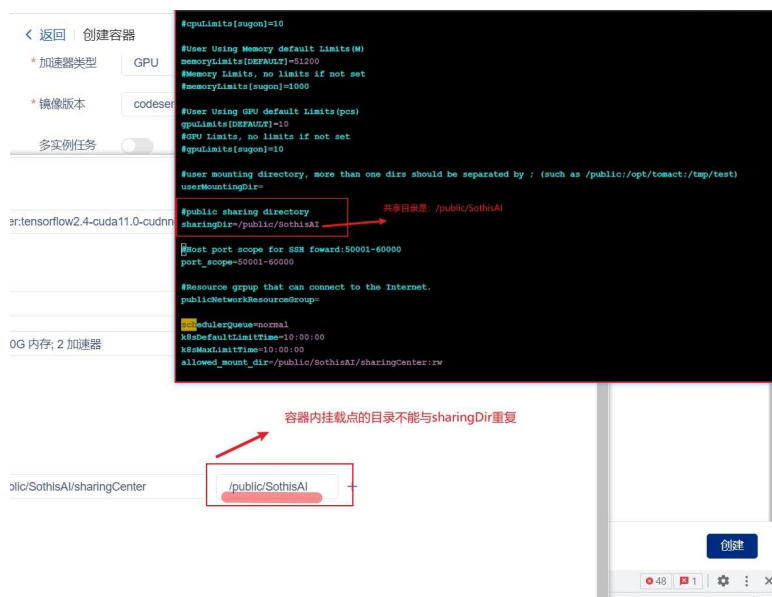
- `${SOTHISAI_HOME}/.gridview/conf/sothisai/scheduler.properties`

2) 文件中的各项配置含义

`allowed_mount_dir=/public/SothisAI/sharingCenter` #允许挂载的目录，多个时用英文;分割，支持权限控制，例如：`/public/SothisAI/sharingCenter:rw;/public/sugon:rw` 表示挂载进去后开启读写权限，默认情况下是 `ro` 只读权限。

`sharingDir=/public/SothisAI` #中心共享目录，默认所有的容器都会将此目录挂载进去，挂载后的目前权限为 `ro` 只读，不支持权限配置。

【注意】:容器内挂载点的目录不能与 `sharingDir` 目录重复，因为容器内不支持重复挂载。如下图：



2.3.7 调整镜像构建节点

默认安装时，镜像库节点和镜像构建节点处于同一节点。若期望将镜像构建节点独立，需保障：

- 1) 镜像构建节点挂载共享存储；
- 2) 镜像构建节点与集群的用户认证系统一致；
- 3) 镜像构建节点最好具备连接公网的能力，以便在镜像构建过程中，可安装相关依赖。

镜像构建节点的调整过程如下：

- 1) 将镜像构建节点的节点名写入到`/opt/build_image_node`文件内；
- 2) 为镜像构建节点安装容器组件；

```
sh /opt/gridview/sothisai_container/batch_install_container_component.sh /opt/build_image_node
```

3) 调整配置文件`/opt/gridview/.gridview/conf/sothisai/image.properties`，将 `buildImageIP` 字段调整为新的镜像构建节点名称。

```
#redisPort=6379
#localCommand Wait for process over time (ms)
timeOut=7200000
#hot images time scope(unit:hour)
hotImagesTimeScope=72
#computer disk threshold(unit:G)
computerDiskThreshold=30
#is this manager node to control clear images(value: "off | on")
clearImageSwitch=off
#sourcefind Switch(value: "off | on")
sourcefindSwitch=on
#sourcefind url
sourcefindUrl=https://sourcefind.cn/#/service-list
#dockerHub url
dockerHubUrl=https://hub.docker.com
#buildImageIP=inner-harbor
buildImageIP=image.oc.com
```